

Veri Yapılarına Giriş

Dr. Öğr. Üyesi Fatma AKALIN

Programlamada veri yapıları en önemli unsurlardan birisidir. Program kodlarını yazarken kullanılacak veri yapısının en ideal şekilde belirlenmesi, **belleğin ve çalışma biçiminin daha etkin** kullanılmasını sağlar.

Veri tipi (data type) ile veri yapısı (data structure) arasındaki ilişkiyi en sade haliyle şöyle açıklayabiliriz:

- ◇ Veri tipi, verinin ne olduğunu
- ◇ Veri yapısı, verinin nasıl organize edildiğini belirtir.

Veri Tipleri

Type	Bits	Range
int	16	-32768 to -32767
unsigned int	16	0 to 65535
signed int	16	-31768 to 32767
short int	16	-31768 to 32767
unsigned short int	16	0 to 65535
signed short int	16	-32768 to -32767
long int	32	-2147483648 to 2147483647
unsigned long int	32	-2147483648 to 2147483647
signed long int	32	0 to 4294967295
float	32	3.4E-38 to 3.4E+38
double	64	1.7E-308 to 1.7E+308
long double	80	3.4E-4932 to 3.4E+4932
char	8	-128 to 127
unsigned char	8	0 to 255
signed char	8	-128 to 127

C++ Veri Tipleri

16-bit Sistemler

int → 2 byte (16 bit)

32-bit Sistemler

int → 4 byte (32 bit)

64-bit Sistemler

Çoğu modern sistemde:

int → hala 4 byte

<https://medium.com/kodcular/algoritma-ve-programlama-d%C3%BCnyas%C4%B1-c-veri-tipleri-739c49e25539>

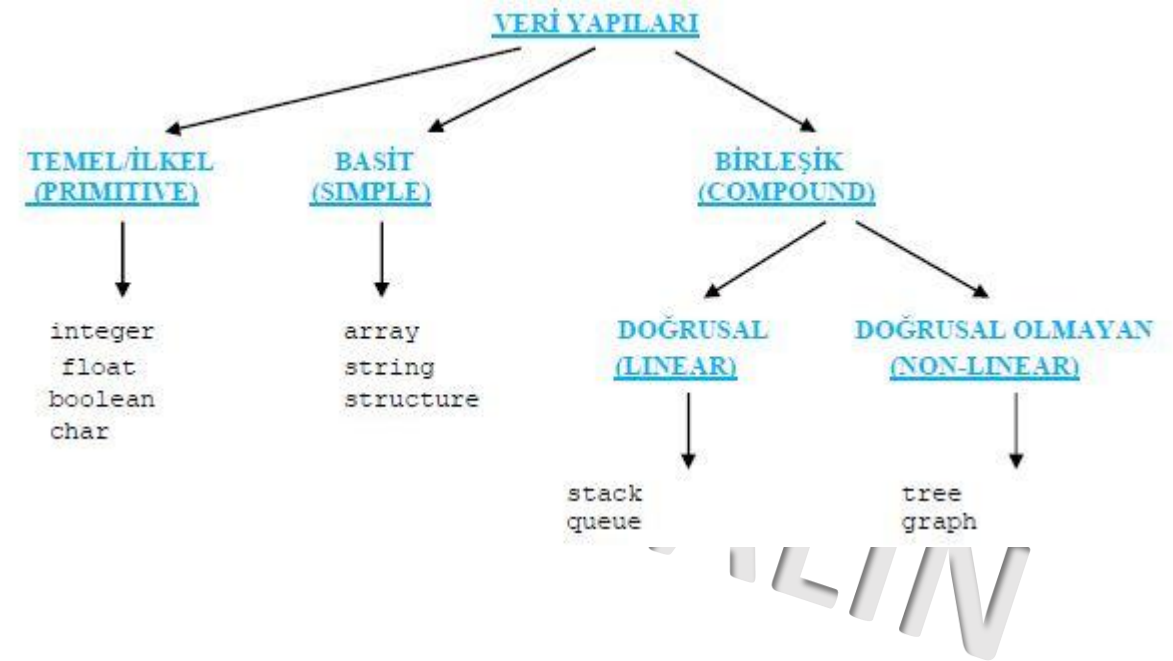
Her programlama dilinin tablodakine benzer, kabul edilmiş veri tipi tanımlamaları vardır. Programcı, programını yazacağı problemi incelerken, program algoritmasını oluştururken, programda kullanacağı değişken ve sabitlerin veri tiplerini bu tanımlamaları dikkate alarak belirler. Çünkü veriler bellekte tablodaki veri tiplerinden kendisine uygun olanlarının özelliklerinde saklanır. **Program, belleğe saklama/yazma ve okuma işlemlerini, işlemci aracılığı ile işletim sistemine yaptırır.** Yani programın çalışması süresince **program, işlemci ve işletim sistemi ile birlikte** iletişim halinde, **belleği** kullanarak işi ortaya koyarlar. Veri için seçilen **tip** bilgisayarın birçok kısmını **etkiler**, ilgilendirir. Bundan dolayı uygun veri tipi seçimi programlamanın önemli bir aşamasıdır. Programcının doğru karar verebilmesi, veri tiplerinin yapılarını tanımasına bağlıdır.

C++ dilinin veri yapıları

C++ Veri Yapıları **Temel/İlkel (primitive)**, **Basit (simple)**, **Birleşik (compound)** olarak üç sınıfta incelenmektedir.

- Temel veri yapıları, en çok kullanılan ve **diğer veri yapılarının oluşturulmasında** faydalanılan yapılardır.
- Basit veri yapıları, temel veri yapılarından faydalanılarak oluşturulan **diziler (arrays)**, **stringler**, **yapılar (structures)** ve **birleşimler (unions)**'dir.
- Birleşik veri yapıları, **temel ve basit veri yapılarından** faydalanılarak oluşturulan diğerlerine göre daha **karmaşık** veri yapılarıdır.

Program, işlemci ve işletim sistemi her veri yapısına ait verileri farklı biçim ve teknikler kullanarak, **bellekte yazma ve okuma işlemleriyle uygulamalara taşırlar**. Bu işlemlere Veri Yapıları Algoritmaları denir. Çeşitli veri yapıları oluşturmak ve bunları programlarda kullanmak programcıya programlama **esnekliği** sağlarken, bilgisayar donanım ve kaynaklarından **en etkin biçimde faydalanma** olanakları sunar, ayrıca programın **hızını ve etkinliğini artırır, maliyetini düşürür**.



Belleğin Yapısı ve Veri Yapıları

Yapısal olarak **bellek**, büyüklüğüne bağlı olarak binlerce, **milyonlarca 1'er Byte (8 Bit)'lik veriler** saklayabilecek biçimde tasarlanmış bir elektronik devredir. Bellek mimarisinde Her bir **hücreyi birbirinden** ayırmak için **hücreler numaralarla** adreslenir. **Program, işlemci ve işletim sistemi bu adresleri kullanarak verilere erişir** (yazar/okur). Büyük olan bu adres numaraları 0'dan başlayarak bellek boyutu kadar sürer ve 16'lık (**Hexadecimal**) biçimde ifade edilir. **İşletim sistemleri belleğin bazı bölümlerini kendi dosya ve programlarını çalıştırmak** için, bazı bölümlerini de **donanımsal** gereksinimler için ayırır ve kullanır. Ancak belleğin büyük bölümü uygulama programlarının kullanımına ayrılmıştır.

Referanslar ve Referans Parametreleri

Fonksiyonlara argüman transferinin iki yolu vardır Bunlar **değer ile geçiş** ve **referans ile geçiştir**. Bir argüman **değer** ile transfer edilirse argüman değerinin bir **kopyası** oluşturulur ve çağırılan fonksiyona geçirilir. **Kopyadaki değişiklikler çağırınının değişken değerini etkilemez**, bu durum yanlışlıkla olan fonksiyon yan etkilerini önler. Değer ile geçişin bir dezavantajı şudur geçirilecek daha çok büyükse bunun fonksiyona geçirilmesi **zaman alıcı ve yer kaplayıcı** olabilmektedir

Referans Parametreleri

Bu kısımda çağrılan fonksiyona çağıran fonksiyonun verilerine doğrudan erişme ve düzenleme yetkisi veren **referans** ile geçiş incelenecektir. Referans ile geçiş **performans açısından iyi** sonuçlar verir Çünkü bu yöntem büyük verilerin değer ile çağırım yöntemindeki **kopyalama yükünü eler.**

Referans parametresi program çağırımında ilgili argümanın gösterilmesi için farklı bir yoldur. Bir fonksiyon parametresinin **referans ile gösterilmesi istenirse**, fonksiyon prototipinde o parametre tipinin hemen devamını **ampersand &** işareti konur. & ile tanımlanan referans parametresi orijinal değişken için bir takma isimdir. Takma isim üzerinde yapılan işlemler aynı zamanda **orijinal değişken üzerinde yapılmış** olur. Fonksiyon başlığında da aynı işlem uygulanır.

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4  int squareByValue(int);
5  void squareByReference(int &);
6  //squareByValue sayıyı kendisi ile çarpar, sonucu bu sayıda tutar
7  //ve sayının yeni değerini döndürür
8  int squareByValue(int number){
9      return number*=number;//çağırılan fonksiyonun argumanı değişmemiştir
10 }
11
12 //squareByReference fonksiyonu numberRef değerini kendisi ile çarpar, ve sonucu
13 //NumberRef'in main fonksiyonu içinde belirttiği değişkene atar
14
15 void squareByReference(int &numberRef){
16     numberRef*=numberRef;//çağırılan fonksiyonun argumanı değişmiştir
17 }
18
19 int main(){
20     int x=2; //squareByValue fonksiyonu ile karesi alınacak deger
21     int z=4; //squareByReference fonksiyonu ile karesi alınacak deger
22     //squareByValue ile çıktıyı görelim
23     cout<<"x= "<<x<<"before squareByValue\n";
24     cout<<squareByValue(x)<<endl;
25     cout<<"x= "<<x<<"after squareByValue\n";
26     //squareByReference ile çıktıyı görelim
27     cout<<"z= "<<z<<"before squareByReference\n";
28     squareByReference(z);
29     cout<<"z= "<<z<<"after squareByReference\n";
30 }

```

(squareByValue): Fonksiyon sadece değeri bir kez işler ve geri döndürür.
Ancak dönen değeri **tekrar kullanmıyorsunuz** ve orijinal değişken üzerinde bir değişiklik yapmıyorsunuz.

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_9>g++ -o main main.cpp
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_9>main
x= 2before squareByValue
4
x= 2after squareByValue
z= 4before squareByReference
z= 16after squareByReference

```



```

int main() {
    int x=2; //squareByValue fonksiyonu ile karesi alınacak deger
    int z=4; //squareByReference fonksiyonu ile karesi alınacak deger
    //squareByValue ile çıktıyı görelim
    cout<<"x= "<<x<<"before squareByValue\n";
    cout<<squareByValue(x)<<endl;
    cout<<"x= "<<x<<"after squareByValue\n";
    //squareByReference ile çıktıyı görelim
    cout<<"z= "<<z<<"before squareByReference\n";
    squareByReference(z);
    cout<<"z= "<<z<<"after squareByReference\n";
}

```

```

C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek10>g++ -o main main.cpp
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek10>main
count= 1
cRef= 1count= 1cRef= 2count= 2

```

C++'da referans değişkenler (&), bir değişkenin takma adı (alias) gibidir. Bellekte ayrı bir alan kaplamazlar ve doğrudan bağlandıkları değişkenle aynı adresi paylaşırlar.

Göstericiler (*) ise adres bilgisini tutar. Referanslar ise adresle uğraşmadan doğrudan değişken gibi davranır.

İşaretçi Değişken Tanımlama ve İlk değer Atama Dolaylama

İşaretçi değişkenler değerlerinde hafıza adreslerini saklarlar.

Normalinde bir değişken doğrudan belirli bir değeri içerir. **Bir işaretçi** belirli değeri içeren bu değişkenin yerine **değişkenin hafızadaki adresini içerir**. Bu bağlamda bir **değişken ismi doğrudan bir değere** başvurur ve bir **işaretçi** ise **dolaylı olarak bir değere** başvurur. İşaretçi yoluyla başvurma dolaylama olarak adlandırılır.

İşaretçi Operatörleri

Adres (&) Operatörü

Adres operatörü (&) operandlarının hafıza adresini alan tekli operatördür. Örneğin;

```
int y=5; //y değişkenini tanımla
```

```
int *yPtr=y; //yPtr değişkeni için işaretçi tanımla
```

Bildirimlerini varsayarak

```
yPtr=&y; //y'nin adresini yPtr'ye ata
```

İfade `y` değişkeninin değerini `yPtr` işaretçi değişkenine atar

☞ `yPtr` = `y`'nin adresini tutar

☞ `*yPtr` = `y`'nin değerini verir

Referans Değişkenler (&) ve Adres Operatörü (&) Arasındaki Fark

C++'ta & operatörü, iki farklı anlam taşır:

1- Referans (&) operatörü → Bir değişkenin takma adını (alias) tanımlamak için kullanılır.

2- Adres (&) operatörü → Bir değişkenin bellekteki adresini almak için kullanılır.

Referans (&) ve Adres Operatörü (&) Arasındaki Temel Farklar

Özellik	Referans (&)	Adres Operatörü (&)
Amaç	Değişken için takma ad (alias) oluşturur	Değişkenin adresini alır
Kullanım Alanı	Değişkenlerle aynı isimle çalışmak için	Göstericilerle (pointers) birlikte kullanılır
Aynı bir bellek alanı kaplar mı?	✗ Hayır (bağlandığı değişkenle aynı adresi kullanır)	✓ Evet (adres bilgisini saklayan değişkenlerde kullanılır)
Örnek Kullanım	<code>int &r = x;</code>	<code>int *p = &x;</code>
Başlangıçtan sonra başka bir değişkene bağlanabilir mi?	✗ Hayır	✓ Evet

Dolaylama (*) Operatörü

Tekli * operatörü genellikle dolaylama operatörü veya referanstan ayırma operatörü olarak bahsedilir. İşaretçi operandının işaret ettiği nesneyi gösteren bir **value** döndürür.

```
cout<<*yPtr<<endl; ifadesi
```

Referanstan Ayrılmış İşaretçi Nedir?

- Referans (&) → Bir değişkenin takma adıdır ve hep aynı değişkeni gösterir.
- İşaretçi (*) → Bir değişkenin adresini tutar ve farklı değişkenlere atanabilir.
 - **"Referanstan ayrılmış işaretçi" → Bir işaretçinin başlangıçta bir referans gibi çalışırken (hep aynı değişkeni gösterirken), daha sonra farklı adreslere atanabilmesi anlamına gelir.**

Özellik	Referans (&)	İşaretçi (*)
Değişkene bağlılık	Hep aynı değişkeni gösterir	Farklı değişkenlere atanabilir
Bellekte ayrı yer tutar mı?	✗ Hayır	✓ Evet
Başka bir değişkene atanabilir mi?	✗ Hayır	✓ Evet
Başka adres gösterebilir mi?	✗ Hayır	✓ Evet
Takma ad gibi mi çalışır?	✓ Evet	✗ Hayır
Bellek adresiyle işlem yapabilir mi?	✗ Hayır	✓ Evet

Adres(&) ve Dolaylama(*) Operatörlerini Kullanma

```
main.cpp x
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int a=7; //a'ya 7 atar
6      int *aPtr=&a; //a tamsayı değişkeninin adresini aPtr ile başlat
7      cout<<"a'nın adresi = "<<&a;
8      cout<<endl;
9      cout<<"aPtr'nin degeri= "<<aPtr;
10     cout<<endl;
11     cout<<"a'nın degeri = "<<a;
12     cout<<endl;
13     cout<<"*aPtr'nin degeri= "<<*aPtr;
14 }
```

```
API\ARI\ders_1\ornek_11>g++ -o main main.cpp
API\ARI\ders_1\ornek_11>main
API\ARI\ders_1\ornek_11>
```

AKALIN

İşaretçiler ile Referans ile Gönderme

C++'da bir fonksiyona üç farklı şekilde argüman gönderilebilir:.

1-Değer ile gönderme

2-Referans argümanları ile **referans** ile gönderme

3-İşaretçi argümanları ile **referans** ile gönderme

Referans parametreleri aynı zamanda programların bir fonksiyona geniş veri nesneleri göndermesine izin verir ve **nesnelerin değer ile gönderilmesinin ek yükünü önler.**

İşaretçi Argümanları ile Referans ile Gönderme:

İşaretçi, bir nesnenin bellekteki adresini tutan bir değişkendir.

Fonksiyona işaretçi ile gönderilen bir argüman, yine orijinal nesne üzerinde değişiklik yapılmasına olanak sağlar.

İşaretçiler, * ve & sembollerini kullanarak tanımlanır. Bir işaretçi fonksiyonu çağırırken, adresi ile gönderilir ve fonksiyon içinde dereference edilerek (yani adresi üzerinden erişilerek) kullanılır.

İşaretçiler nullptr gibi geçersiz değerler alabilir, bu yüzden işaretçi geçmeden önce geçerli olup olmadığını kontrol etmek önemlidir.

* → pointer tanımlamak ve dereference etmek için kullanılır.
& → adres almak için kullanılır.

Referans Argümanları ile Referans ile Gönderme:

Bir referans, aslında bir değişkenin kendisini gösteren bir "takma ad"dır.

Fonksiyona referans ile gönderilen bir argüman, fonksiyonda orijinal değişkeni değiştirebilir.

Referanslar, & sembolü ile tanımlanır ve fonksiyon çağrısı sırasında argüman değerinin bir kopyası değil, kendisi gönderilir.

nullptr veya null değerleriyle çalışamazlar, yani her zaman geçerli bir nesneye işaret ederler.

Değer ile Gönderme

Dr. Öğr. ...

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_12>g++ -o main main.cpp
```

```
main.cpp x
1  #include <iostream>
2  using namespace std;
3
4  int cubeByValue(int); //prototip
5  int main() {
6      int number=5;
7      cout<<"Sayinin orijinal degeri: "<<number<<endl;
8      number=cubeByValue(number); //sayıyı cubeByValue fonk. değer ile gönderme
9      cout<<"Sayinin yeni degeri: "<<number<<endl;
10 }
11 int cubeByValue(int n){
12     return n*n*n; //n lokal değişkeninin küpünü hesapla ve sonucu döndür
13 }
```

(cubeByValue): Fonksiyon, değeri işler ve geri döndürdüğü sonucu doğrudan orijinal number değişkenine atıyorsunuz. Bu nedenle number değişiyor ve 125 oluyor.

İşaretçi Kullanılarak Referans ile Gönderme

* → "Ben adres tutuyorum" (pointer)& → "Bu değişkenin adresini al"
Yani:Fonksiyon adres istiyor
(int *)Biz adres gönderiyoruz (&number)

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_13>g++ -o main main.cpp  
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_13>main
```

```
main.cpp x  
1  #include <iostream>  
2  using namespace std;  
3  
4  void CubeByReference(int *); //prototip  
5  int main(){  
6      int number=5;  
7      cout<<"Sayinin orijinal degeri: "<<number<<endl;  
8      CubeByReference(&number); //CubeByReference'a sayinin adresini gönderme  
9      cout<<"Sayinin yeni degeri: "<<number<<endl;  
10 }  
11 void CubeByReference(int *nPtr){  
12     *nPtr= *nPtr * *nPtr * *nPtr; //n lokal değişkeninin küpünü hesapla ve sonucu döndür  
13 }
```

Burada işaretçi nPtr, number değişkeninin adresini gösteriyor.*nPtr ifadesi, işaretçinin gösterdiği yerden değeri alır, yani number değişkeninin değerini.*nPtr * *nPtr * *nPtr ifadesi, number'ın küpünü hesaplar ($5 * 5 * 5 = 125$) ve bu yeni değeri number'a atar.

!!UYARI!!

Bir fonksiyonun başlığında **sabit bir dizinin parametresini** şu şekilde tanımlayabiliriz.

```
int sumElements(const int values[], const size_t numberOfElements)
```

Bu tanımlama, fonksiyonun **birinci argümanının** fonksiyon tarafından değiştirilemeyecek tam sayılardan oluşan tek boyutlu sabit boyutlu dizi olduğunu gösterir. Array nesnelerinin aksine sabit boyutlu diziler sahip oldukları boyutu bilmezler yani sabit boyutlu diziye işleyen fonksiyon hem sabit boyutlu diziye hem de boyutunu içeren parametreler almalıdır. Yukarıda gösterilen başlık **şu şekilde de yazılabilir.**

```
int sumElements(const int *values, const size_t numberOfElements)
```

Derleyici **işaretçi alan bir fonksiyon** ile **sabit boyutlu dizi alan bir fonksiyon** arasındaki farkı ayırt etmez. Derleyici **const int values[]** şeklinde tek boyutlu sabit bir dizi için fonksiyon parametresi ile karşılaştığında, derleyici bu parametreyi **const int values*** işaretçi biçimine dönüştürür. Yani values bir tamsayı sabiti işaretçisidir.

Sabit olmayan Veri İçin Sabit Olmayan İşaretçi

En yüksek erişim sabit olmayan veri için sabit olmayan işaretçi tarafından yerine getirilir. Bu şekilde işaretçi tanımlamaları `const` **içermez**(örneğin, `int *countPtr`).

Dr. Öğr. Üyesi Fatma AKALIN

Sabit veri için sabit olmayan işaretçi uygun tipteki herhangi bir veri ögesine işaret etmek için değiştirilebilecek fakat işaret ettiği **verinin işaretçi yoluyla değiştirilemeyeceği bir işaretçidir**. Bu şekilde bir işaretçi, elemanlarını okumasına izin veren fakat değiştirmesine izin vermeyen bir fonksiyon için sabit boyutlu dizi argümanları almak amacıyla kullanılabilir. Fonksiyondaki verileri değiştirmeye çalışmak bir derleme hatası ile sonuçlanır. Bu şekilde işaretçi bildiriminde aşağıda olduğu gibi işaretçi tipinin soluna const yerleştirilir.

```
const int *countPtr;
```

Yani, **countPtr bir tamsayı sabiti için sabit olmayan bir işaretçidir.**

Yani, burada, countPtr bir işaretçi (*countPtr), bir int türüne işaret eder. Sabit veri, const kelimesi, işaret edilen verinin değiştirilemeyeceğini belirtir. Yani, countPtr işaret ettiği veriye değişiklik yapamaz. Sabit olmayan işaretçi: İşaretçinin kendisi değiştirilebilir. Yani, countPtr işaret ettiği yeri (adresini) değiştirebilir, başka bir int değerine işaret edebilir.

Sabit bir veri için sabit olmayan işaretçi alan, daha sonra da bu veriyi değiştirmek için işaretçi kullanmaya çalışan bir fonksiyonu derlemeye çalışırken oluşan GNU C++ derleme hata mesajını inceleyelim.

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_14>g++ -o main main.cpp

main.cpp
1  #include <iostream>
2  using namespace std;
3
4  void f(const int *); //prototip
5  int main(){
6      int y=0;
7      f(&y); //f, geçersiz bir değıştirmeye kalkışabilir
8  }
9  void f(const int *xPtr){
10     *xPtr= 100; //hata:const tanımlı nesneyi değıştiremez
11 }
```

y location '* xPtr'
değıştiremez

Adres bilgisi geçiriliyor.

xPtr o adres bilgisini tutuyor.

*xPtr adres bilgisindeki veriye dolaylı erişir.

Sabit Olmayan Veri İçin Sabit İşaretçi

Sabit olmayan veri için sabit işaretçi her zaman **aynı hafıza alanına işaret eden bir işaretçidir** ve bu konumdaki **veri işaretçi yoluyla değiştirilebilir**. `const` olarak tanımlanan işaretçiler tanımlandıklarında başlatılmalıdır. Fakat işaretçi bir fonksiyon parametresi ise fonksiyona gönderilen işaretçi ile birlikte başlatılır.

AKALIN

Örnekte ptr, **int * const** tipinde tanımlanır. Tanımlama, ptr **sabit olmayan bir tamsayı için sabit bir işaretçidir**.

```
main.cpp x
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int x,y;
6      int * const ptr=&x; //const işaretçiye ilk değer atanmalıdır.
7      *ptr=7; //izin verilir: *ptr const değildir
8      ptr=&y; //hata:ptr, consttur:yeni bir adrese atanamaz
9  }
```

```
C:\Users\Fatma\Desktop\VERİ YAPILARI\ders_1\ornek_15>g++ -o main main.cpp
main.cpp: In function 'int main()':
main.cpp:8:7: error: assignment of read-only variable 'ptr'
    ptr=&y; //hata:ptr, consttur:yeni bir adrese atanamaz
    ^
```

const soldaysa → değer sabit
const sağdaysa → pointer sabit

Sabit Veri İçin Sabit İşaretçi

Özel izin erişimi sabit veri için sabit işaretçi tarafından yerine getirilir. Bu tarz bir **işaretçi aynı hafıza alanına** işaret eder ve bu konumdaki **veri işaretçi yoluyla değiştirilemez**. Bu sabit bir dizinin sabit boyutlu diziden dizi indeks notasyonunu kullanarak sadece okuyan ve sabit boyutlu diziyi değiştirmeyen fonksiyona gönderilmesi gibidir.

```
main.cpp x
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int x=5,y;
6      const int *const ptr=&x;
7      cout<<*ptr<<endl;
8      *ptr=7; //hata:*ptr sabittir; yeni değer atanamaz
9      ptr=&y; //hata:ptr sabittir; yeni adrese atanamaz
10 }
```

```
ders_1\ornek_16>g++ -o main main.cpp
```

```
error: '*(const int*)ptr' is a read-only location
error: 'ptr' is a read-only variable
error: cannot assign to 'ptr'
error: cannot assign to 'ptr'
error: cannot assign to 'ptr'
```

sizeof Operatörü

sizeof derleme zamanı tekli operatörü program derlemesi boyunca sabit bir dizinin veya başka bir veri tipinin, değişkenin veya sabitin **boyutunu bayt** olarak belirler.

Dr. Öğr. Üyesi Fatma AKALIN

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_17>g++ -o main main.cpp
```

```
C:\Users\Fatma\Desktop\VERİYAPILARI\ders_1\ornek_17>main
```

```
sizeof c=1
```

```
sizeof char=1
```

```
sizeof s=2
```

```
sizeof short=2
```

```
sizeof i=4
```

```
sizeof int=4
```

```
sizeof l=4
```

```
sizeof long=4
```

```
sizeof ll=8
```

```
sizeof long long=8
```

```
sizeof f=4
```

```
sizeof float=4
```

```
sizeof d=8
```

```
sizeof double=8
```

```
sizeof ld=12
```

```
sizeof long double=12
```

```
sizeof array=80
```

```
sizeof ptr=4
```

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main(){
5      char c;
6      short s;
7      int i;
8      long l;
9      long long ll;
10     float f;
11     double d;
12     long double ld;//long double tipinde bir degisken
13     int array[20];//int tipinde sabit boytlu bir dizi
14     int *ptr=array;//int * tipinde bir degisken
15
16     cout<<"sizeof c="<<sizeof c<<endl;
17     cout<<"\nsizeof char="<<sizeof(char)<<endl;
18     cout<<"\nsizeof s="<<sizeof s<<endl;
19     cout<<"\nsizeof short="<<sizeof (short)<<endl;
20     cout<<"\nsizeof i="<<sizeof i<<endl;
21     cout<<"\nsizeof int="<<sizeof (int)<<endl;
22     cout<<"\nsizeof l="<<sizeof l<<endl;
23     cout<<"\nsizeof long="<<sizeof (long)<<endl;
24     cout<<"\nsizeof ll="<<sizeof ll<<endl;
25     cout<<"\nsizeof long long="<<sizeof (long long)<<endl;
26     cout<<"\nsizeof f="<<sizeof f<<endl;
27     cout<<"\nsizeof float="<<sizeof (float)<<endl;
28     cout<<"\nsizeof d="<<sizeof d<<endl;
29     cout<<"\nsizeof double="<<sizeof (double)<<endl;
30     cout<<"\nsizeof ld="<<sizeof ld<<endl;
31     cout<<"\nsizeof long double="<<sizeof (long double)<<endl;
32     cout<<"\nsizeof array="<<sizeof array<<endl;
33     cout<<"\nsizeof ptr="<<sizeof ptr<<endl;
34 }
```

16-bit Sistemler
int → 2 byte (16 bit)

32-bit Sistemler
int → 4 byte (32 bit)

64-bit Sistemler

Çoğu modern sistemde:
int → hala 4 byte

KALIN

İşaretçi İfadeleri ve İşaretçi Aritmetiği

İşaretçi aritmetiği C++'ta mümkündür ve belirli kurallara bağlı olarak işaretçiler üzerinde çeşitli işlemler yapılabilir. Ancak, işaretçi aritmetiği yalnızca sabit boyutlu dizilerle sınırlıdır ve şu işlemler yapılabilir:

- **İşaretçiyi artırma veya azaltma:** İşaretçi, gösterdiği elemanı takip eden veya önceki elemana işaret edecek şekilde artırılabilir veya azaltılabilir.
- **İşaretçi ile tam sayı ekleme veya çıkarma:** Bir işaretçiye bir tam sayı eklenebilir veya çıkartılabilir. Bu, işaretçinin gösterdiği elemanın tipine bağlı olarak, işaretçinin gösterdiği adresi kaydırır.
- **İki işaretçi arasındaki fark:** Aynı türdeki iki işaretçi arasındaki fark hesaplanabilir. Bu işlem, işaretçilerin hangi dizinin elemanlarını işaret ettiğine göre anlamlıdır.

Önemli Nokta

İşaretçi aritmetiği yalnızca **aynı sabit boyutlu dizinin elemanlarını işaret eden** işaretçiler arasında geçerlidir. İşaretçiler farklı dizilere işaret ediyorsa, bu tür işlemler belirsiz sonuçlar verebilir.


```
int *vPtr=v;  
int *vPtr=&v[0];
```

int *vPtr = v; ve int *vPtr = &v[0];
ifadeleri aslında aynı şeyi ifade eder, çünkü her iki durumda da işaretçi vPtr dizinin ilk elemanını işaret eder.

İşaretçilere Tamsayı Ekleme ve Çıkartma İşaretçileri Çıkartma

```
vPtr+=2;  
vPtr-=4;  
++vPtr;  
vPtr++;  
--vPtr;  
vPtr--;  
X=v2Ptr-vPtr;
```

Yandaki ifadeler, işaretçi aritmetiği ile ilgili işlemleri içermektedir. İşaretçi aritmetiği, **işaretçilerin işaret ettiği bellek adresleri üzerinde işlem yapmanızı** sağlar. Ancak burada dikkat edilmesi gereken bazı önemli noktalar vardır. Şimdi her bir ifadeyi detaylı olarak açıklayalım:

1. vPtr += 2;

Açıklama: Bu ifade, işaretçi vPtr'yi, işaret ettiği elemandan iki eleman ileriye kaydırır. Yani, vPtr işaretçisini v[0]'dan v[2]'ye taşır.

Örnek: Eğer vPtr başlangıçta v[0]'ı işaret ediyorsa, bu işlem sonrası v[2]'yi işaret eder.

2. vPtr -= 4;

Açıklama: Bu ifade, işaretçi vPtr'yi, işaret ettiği elemandan dört eleman geri kaydırır. Yani, vPtr işaretçisini 4 eleman geriye alır.

Örnek: Eğer vPtr şu anda v[2]'yi işaret ediyorsa, bu işlem sonrası v[0]'ı işaret eder.

3. ++vPtr;

Açıklama: Bu ifade, vPtr işaretçisini bir eleman ileri kaydırır. Yani, vPtr işaretçisini şu anki elemandan bir sonraki elemanına yönlendirir.

Örnek: Eğer vPtr şu anda v[0]'ı işaret ediyorsa, ++vPtr işlemi v[1]'e taşır.

4. vPtr++;

Açıklama: Bu ifade, vPtr işaretçisini bir eleman ileri kaydırır. ++vPtr ile aynı işlevi görür, ancak burada işlem sonrası vPtr önceki değeri döndürür (post-increment).

Örnek: Eğer vPtr şu anda v[0]'ı işaret ediyorsa, işlem sonrası v[1]'i işaret eder.

5. --vPtr;

Açıklama: Bu ifade, vPtr işaretçisini bir eleman geri kaydırır. Yani, vPtr işaretçisini şu anki elemandan bir önceki elemanına yönlendirir.

Örnek: Eğer vPtr şu anda v[2]'yi işaret ediyorsa, --vPtr işlemi v[1]'e taşır.

6. vPtr--;

Açıklama: Bu ifade, vPtr işaretçisini bir eleman geri kaydırır. --vPtr ile aynı işlevi görür, ancak burada işlem sonrası vPtr önceki değeri döndürür (post-decrement).

Örnek: Eğer vPtr şu anda v[2]'yi işaret ediyorsa, işlem sonrası v[1]'i işaret eder.

7. X = v2Ptr - vPtr;

Açıklama: Burada, iki işaretçi arasındaki fark hesaplanmaktadır. İşaretçilerin farkı, dizinin elemanlarının sayısı olarak hesaplanır. Yani, v2Ptr ve vPtr arasındaki fark, bu işaretçilerin işaret ettiği dizinin eleman sayısıdır.

Örnek: Eğer vPtr şu anda v[1]'i işaret ediyorsa ve v2Ptr v[4]'ü işaret ediyorsa, bu işlem $4 - 1 = 3$ sonucunu verir. v2Ptr, vPtr'den 3 eleman ileridedir.

Not: vptr++-> Önce mevcut değer kullanılır

Sonra azaltılır/arttırılır

Önce *vPtr okunur

Sonra vPtr bir geri/ileri gider

İşaretçi Atama

Bir işaretçi, eğer **her iki işaretçide aynı tipte ise başka bir işaretçiye atanabilir**. Aksi halde, atamanın **sağ tarafındaki işaretçinin değeri, sol tarafındaki işaretçinin tipine** dönüştürmek için bir **tip dönüşümü** operatörü kullanılır.

Bu kuralın **istisnası**, herhangi bir veri tipini gösterebilen **generic işaretçi olan pointer to void**(yani, **void ***) kullanmaktır. Temel veri veya sınıf tipindeki **herhangi bir işaretçi**,tip dönüşümü olmadan **void * tipindeki bir işaretçiye** atanabilir.

Bir void *Referanstan Ayrılamaz

Bir void* işaretçi, referanstan ayrılamaz. Örneğin derleyici, tamsayılar için 4 bayt tutan bir makinede **int için işaretçinin hafızada 4 byte** tuttuğunu bilir. Bununla birlikte **void** bir işaretçi void* işaretçisi, genellikle bir tür işaretçisiyle aynı bellek boyutuna sahip olur. Yani, int* veya char* gibi işaretçilerle aynı boyutta olur (yani genellikle 4 veya 8 bayt, platforma bağlı olarak).

void* işaretçisi bir türün adresini tutmaz, sadece bir bellek adresini saklar. Bu yüzden void* işaretçisi ile doğrudan bellek aritmetiği yapılamaz. Örneğin, bir void* işaretçisini artırmaya çalışırsanız, derleyici hata verir çünkü void* işaretçisi herhangi bir veri türüne işaret etmediği için artırma işleminde hangi türün büyüklüğüne göre artırılacağını bilmez.

İşaretçileri Karşılaştırma

İşaretçiler, eşitlik ve ilişkisel operatörler kullanılarak karşılaştırılabilir.

İlişkisel operatörler kullanarak karşılaştırmalar, aynı sabit boyutlu dizinin **elemanlarını** işaret eden işaretçiler olmadan anlamsızdır.

İşaretçi karşılaştırmaları işaretçilerde saklanan adresleri karşılaştırırlar. Bir sabit boyutlu dizi, bellekte ardışık elemanlar içerdiği için, bu dizinin her elemanının adresi birbirine yakındır. Yani, bir işaretçi ilerledikçe dizinin daha sonraki elemanlarına işaret eder. İşaretçilerin karşılaştırılması, bu dizide hangi elemanın daha önce olduğunu veya daha sonra olduğunu gösterir.

Ayrıca İşaretçi karşılaştırmamanın bir diğer kullanımı bir işaretçi değerinin nullPtr, 0 veya null olup olmadığını belirlemektir.

İşaretçiler ve Sabit boyutlu diziler arasındaki ilişki

Sabit boyutlu diziler ve işaretçiler C++ dilinde çok yakın bir ilişkiye sahiptir ve belirli durumlarda birbirlerinin yerine kullanılabilirler. Ancak, aralarında bazı önemli farklar da vardır. Aşağıdaki tanımları inceleyelim.

```
int b[5]; //5 elemanlı bir b dizisi oluştur;
```

```
int *bPtr; //int tamsayı bir bPtr işaretçisi oluştur
```

```
bPtr=b; //bPtr'ye b sabit boyutlu dizinin ilk elemanının adresini ata
```

bPtr'ye b sabit boyutlu dizinin ilk elemanının adresini **bPtr=b;** ifadesi ile atayabiliriz. Bu, ilk elemanının adresini aşağıdaki gibi atama ile aynıdır.

```
bPtr=&b[0] // bPtr'ye b sabit boyutlu dizinin adresini ata
```

Her ne kadar sabit boyutlu diziler ve işaretçiler bellek adresi üzerinden benzer şekilde erişim sağlasa da, dizi bir veri bloğunu temsil ederken işaretçi yalnızca bir adres bilgisini tutan bağımsız bir değişkendir.

İşaretçi/Ofset Gösterimi

Sabit boyutlu dizi elemanı **b[3]**, alternatif olarak işaretçi ifadesi ile de gösterilebilir.

***(bPtr+3)**

Yukarıda gösterilen ifade bir işaretçi ofsetidir. **İşaretçi, sabit boyutlu dizinin başlangıcını gösterdiğinde, ofset, hangi sabit boyutlu dizi elemanının gösterileceğini belirtir ve ofset değeri index değeri ile aynıdır.** Bu gösterim işaretçi/ofset gösterimi olarak adlandırılır.

Parantezler gereklidir. Çünkü * operatörünün önceliği + operatörününkinden daha yüksektir.

İşaretçiler ve Sabit Boyutlu Diziler Arasındaki İlişkiyi Örnek ile Açıklama

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int b[] = {10,20,30,40}; //4 elemanlı sabit b dizisini oluştur
6      int *bPtr=b; //bPtr'yi b sabit dizisine işaret edecek şekilde ata
7
8      //dizi indeks gösterimi kullanarak b sabit dizisini goster
9      for(size_t i=0; i<4; ++i)
10         cout<<"b["<<i<<"]="<<b[i]<<"\n";
11
12     //dizi ismi ve işaretçi/ofset gösterimini kullanarak b dizisini goster
13     for(size_t offset1=0; offset1<4; ++offset1)
14         cout<<"b["<<offset1<<"]="<<*(b+offset1)<<"\n";
15
16     //bPtr ve indeks gösterimini kullanarak b sabit dizisini goster
17     for(size_t j=0; j<4; ++j)
18         cout<<"bPtr["<<j<<"]="<<bPtr[j]<<"\n";
19
20     //bPtr ve işaretçi/ofset gösterimini kullanarak b dizisini goster
21     for(size_t offset2=0; offset2<4; ++offset2)
22         cout<<"* (bPtr+"<<offset2<<"]="<<*(bPtr+offset2)<<"\n";
23 }
```

```
a\Desktop\VERİYAPILARI\ders_1\ornek_18>g++ -o main main.cpp
a\Desktop\VERİYAPILARI\ders_1\ornek_18>main
```

b dizisinin başlangıç adresi, dizinin ilk elemanını işaret ettiği için, $b + \text{offset1}$ ifadesi aslında dizinin başlangıcına offset1 kadar ekleme yaparak o elemanın adresini hesaplar.

$$*(b + 0) = b[0]$$

$b + \text{offset1}$ ifadesi, dizinin başlangıç adresine ($b[0]$ 'ın adresi) offset1 kadar ekleme yaparak, dizinin o elemanın adresini hesaplar.

bPtr dizisinin başlangıç adresini tutar, yani bPtr şu an $b[0]$ 'ın adresini gösterir.

$b[]$: 4 elemanlı bir sabit boyutlu dizi tanımlanır. Bu dizinin elemanları sırasıyla 10, 20, 30, ve 40'tır.

$\text{int } *bPtr = b$; bPtr, dizinin ilk elemanını işaret eden bir işaretçi olarak atanır. Yani, bPtr işaretçisi $b[0]$ 'ı gösterir.

BELLEK YÖNETİMİ

Bilgisayar üzerinde bir **veri yapısını gerçekleştirebilmek** için bilgisayar **belleğinin** kullanımına ihtiyaç vardır. Çalışan herhangi bir programdaki **değişkenler, sınıflar, metotlar mutlaka bellekte bir yerde tutulurlar.** Bellekte tutuldukları yerler bakımından 3 farklı bölge bulunmaktadır.

1-Statik Bellek Bölgesi

2-Çalışma Anı Yığını

3-Heap Bellek Bölgesi

Statik Bellek Bölgesi

Bu bölgede yer alacak değişkenlerin hangileri olduğu, **daha program başlamadan** bellidir. Bu bölgede **Global değişkenler, sabitler, static olarak tanımlanmış lokal değişkenler tutulurlar**. Statik bellek bölgesinde tutulan değişkenler program çalıştığı sürece var olurlar, program sonlandığında bellekten silinirler.

Aşağıdaki kod örneğinde **kontrol değişkeni global değişkendir**. **pi sayısı const ifadesi olduğu için sabittir** ve Arttir metodunun içerisindeki **sayim değişkeni de başında static olduğu için statik lokal değişkendir**. Bu ismi anılan 3 değişkende statik bellek bölgesinde tutulur. **Statik bellek bölgesinde tutulduğu için program sonlanıncaya kadar bellekte tutulurlar**. Bundan dolayı Arttir metodu her çağrıldığında sayim değişkeni sıfırdan başlamak yerine kalmış olduğu **değerden devam** edecektir.

Global değişkenler bu bölgede tutuldukları için program sonlanıncaya kadar **bellekte tutulacaktır ve programın herhangi bir satırından bu değişkenlere erişilebilecektir**. İşte bu yüzden global **değişkenlerin kullanımı** (değişip değişmediklerinin kontrolü zor olduğu için) **tavsiye edilmemektedir**.

Kod 1:

```
int kontrol;  
const float pi=3.14;  
int Arttir(){  
    static int sayim = 0;  
    sayim++;  
    return sayim;  
}  
int main(){  
    cout<<Arttir()<<endl;  
    cout<<Arttir()<<endl;  
    cout<<Arttir()<<endl;  
}
```

Çalışma Anı Yığını

En aktif bellek bölgesidir denilebilir. İsmi de oradan aldığı bu bellek bölgesi bir yığın (stack) şeklindedir ve bu yapıda çalışır. **Bu yapıya ilk giren en son çıkar.** Bir program çalıştığı sürece genişleyip daralan bitişik bir yapıya sahiptir. Bu bellek bölgesinde **fonksiyon ve metot çağrımları ve bu fonksiyon ve metotların barındırdığı lokal değişkenler** bulunur. Bir fonksiyon veya metot **çağrıldığında** bu fonksiyon veya metoda ait **parametreler değişkenler ve dönüş değerleri** bu bellek bölgesinde tutulur. **Çalışma anı yığın bölgesi genişlemiş olur. Fonksiyon veya metot çağrılan yere döndüğünde** bu fonksiyon veya metodun çalışma anı yığnında ayırmış olduğu **yer geri döndürülür.** Dolayısıyla geri döndürülen bu değişkenlere **çağırım bittikten sonra erişim olamayacaktır.**

Yukarıdaki kod örneğine bakıldığında, iki metot ve bir main metodu yer almaktadır. Main metodunda **iki kere DegerArttir metodu çağrılmış** ve daha sonra **topla metodu** çağrılmıştır. DegerArttir metodunun içerisindeki sayaç değişkeni **statik lokal değişken olduğu için statik bellek bölgesinde diğer bütün değişkenler, çalışma anı yığnında oluşturulur**. Topla metodunun **çağrımı bittikten sonra**, çalışma anı yığnında oluşturulmuş olan a, b ve sonuc değişkenleri bellekten **yok edilirler**.

Kod 3:

```
int DegerArttir(){
    static int sayac=0; // Statik bellek bölgesinde
    return ++sayac;
}
int topla(int a,int b){
    int sonuc = a+b; // Çalışma anı yığnında
    return sonuc;
}
int main(){
    cout<<DegerArttir()<<endl;
    cout<<DegerArttir()<<endl;
    cout<<topla(21,10)<<endl;
    cout<<topla(5,7)<<endl;
    return 0;
}
```

Çıktı

1

2

31

12

Kod 4:

```
int sayi = 0;

int Arttir(int x){
    return ++x;
}

int main()
{
    cout<<Arttir(sayi)<<endl;
    cout<<sayi<<endl;
    return 0;
}
```

Çıktı
1
0

Yukarıdaki kod örneği incelendiğinde bir **sayi** adında **global değişken** tanımlanmıştır. Main metodu içerisinde **Arttir metodu çağrılıyor** ve daha sonra **sayi** ekrana yazdırılıyor. Arttir metodu parametre olarak aldığı değeri bir arttırıp geri döndüren bir metottur. Bu kodda sayi parametre olarak gönderiliyor, sayi global değişken olsa bile Arttir metodu içerisinde değeri değiştirilen **lokal x değişkenidir** dolayısıyla **çağırım bittikten sonra bellekten silinecek** fakat **sayi değişkeni değerini yani sıfırı koruyacaktır**.

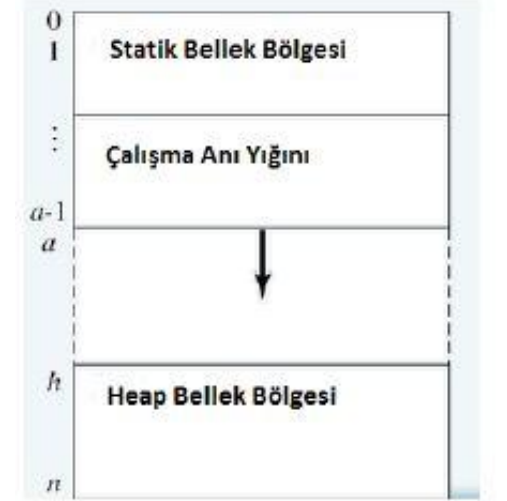
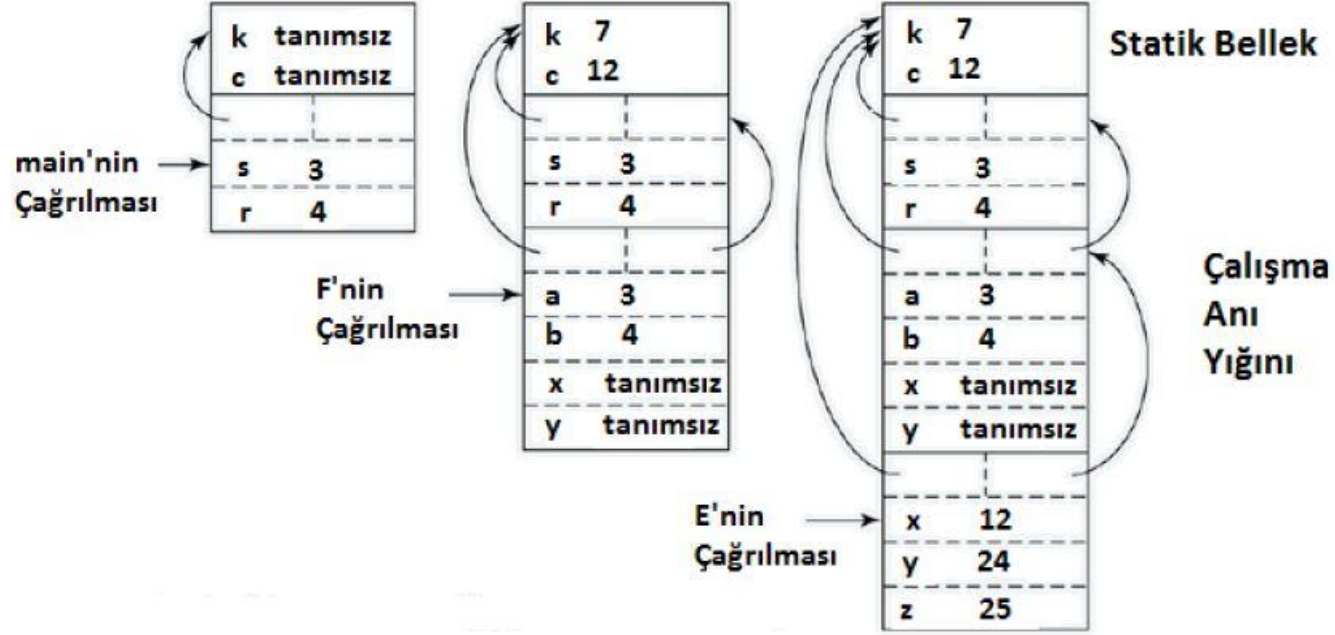
Kod 5:

```
int k,c;

void E(int x){
    int y = 2*x;
    int z = y + 1;
}

void F(int a,int b){
    int x,y;
    c=a*b;
    E(c);
}

int main(){
    int s=3,r=4;
    k=s+r;
    F(s,r);
}
```



Yukarıdaki kod bloğunda, **k ve c değişkenleri global değişkenler, diğer bütün değişkenler lokal değişkenlerdir.**

Main metodundan çağırılma ve değer atanma sırasına bakıldığında bellekteki görüntü aşağıdaki gibi olacaktır.

Şekilde gösterilen çalışma anı yığını bellek bölgesidir. Fakat bellek bölgesinin üst tarafı statik bellek bölgesi olarak gösterilmiştir. Sol taraftan çıkan oklar statik çağırım. Sağ taraftan çıkan kollar ise dinamik çağırımdır.

Örnekteki **dinamik çağırımlar metot çağırımları. Statik çağırımlar ise global değişkenlerin çağırımını gösterir.**

Yığın aşağıya doğru genişliyor bu demek oluyor ki yığının baş tarafı aşağı taraf çünkü yığında ilk giren son çıkar prensibi olduğu için bu şekilde olmak durumunda.

Heap Bellek Bölgesi

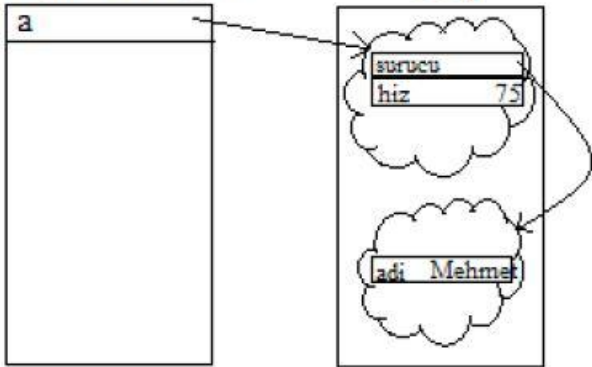
Bu bellek bölgesi C ve C++ gibi programlama dillerinde dikkat edilmesi gereken çok önemli bir bölgedir. Çünkü **C ve C++ gibi dillerde bu bölgenin kontrolü programcıya** bırakılmıştır. Bu da demek oluyor ki eğer bu bölgenin kontrolü iyi sağlanmaz ise **bellek taşması ya da yanlış değerlere erişim gibi problemler** ile karşı karşıya kalınabilir. Bu bölgeye duyulan ihtiyacın nedeni, dinamik oluşturulan yapıların boyutları değişken olacak ve çalışma anında belirlenecektir. Dolayısıyla bu yapılar **heap bellek** bölgesinde **tutulmalı**, bu **yapılara ve değişkenlere göstericiler yardımıyla** erişilmelidir. Bu bellek bölgesinde tutulan bir değer **adı yoktur** yani anonimdir ve ancak değer **bulunduğu adresi** gösterecek bir gösterici yardımıyla erişilebilir.

Heap bellek bölgesinde C++ programlama dilinde bir yer ayırmak için **new** operatörü kullanılır. new operatörü kullanan göstericinin tutulduğu yer **çalışma anı yığınıdır**. Gösterdiği adres ise **heap bellek bölgesinde bulunur**. new operatörü kullanmadan tanımlanan göstericiler heap bellek bölgesinden **yer ayıramazlar**.

Bu bölgede ayrılan yer işi bittiğinde belleğe geri verilmelidir. Bu bölgenin kontrolü programcıda olduğu için **eğer geri döndürülmez ise çöp dediğimiz durum oluşur**. Hatırlanırsa bu bölgedeki adreslere çalışma anı yığınındaki göstericiler yardımıyla erişiliyordu. Dolayısıyla **çalışma anı yığınındaki gösterici kaybedilmeden önce heap bellek bölgesinde ayrılan yer geri döndürülmelidir**. Yoksa o bölge bilgisayar kapanıncaya kadar kullanılamaz duruma gelir.

Yandaki kod örneğinde araç sınıfından heap bellek bölgesinde bir nesne oluşturuluyor. Fakat dikkat edilirse Arac sınıfının yapıcı metodunda bir new operatörü daha var. Burada aracın sürücüsü olan Kişi nesnesi oluşturuluyor. Bu durumda Arac geri döndürülmeden içinde oluşturulmuş olan ve heap bellek bölgesinde bulunan sürücünün gösterdiği kişi nesnesi geri döndürülmesi gerekiyor. İşte bundan dolayı Arac sınıfının yıkıcı metodunda o alan geri döndürülüyor.

Çalışma Anı Yığını



delete a;
çağrıldığında:

- Arac nesnesi heap'ten silinir.
- Ama surucu işaret ettiği Kisi nesnesi silinmez.
- Artık o nesneye erişen hiçbir pointer yoktur.

Bu yüzden destructor içinde:delete surucu;yazılmıştır.

a, normal bir **yerel değişken tanımıdır**.

Kod 7:

```
class Kisi{
private:
    string adi;
public:
    Kisi(string ad):adi(ad){ }
};

class Arac{
private:
    Kisi *surucu;
    float hiz;
public:
    Arac(string surucuAdi,float hz){ //Yapıcı fonksiyon
        surucu = new Kisi(surucuAdi);
        hiz=hiz;
    }
    ~Arac() //Yıkıcı fonksiyon.
    {
        delete surucu;
    }
};

int main()
{
    Arac *a = new Arac("Mehmet",75);
    delete a;
    return 0;
}
```

heap'te bir Kisi nesnesi oluşturuyor.
☞ Bu nesnenin sorumluluğu Arac'a ait.
☞ Yani Arac, surucu'nun sahibidir (owner).
Bu yüzden destructor içinde:delete surucu;yazılmıştır.

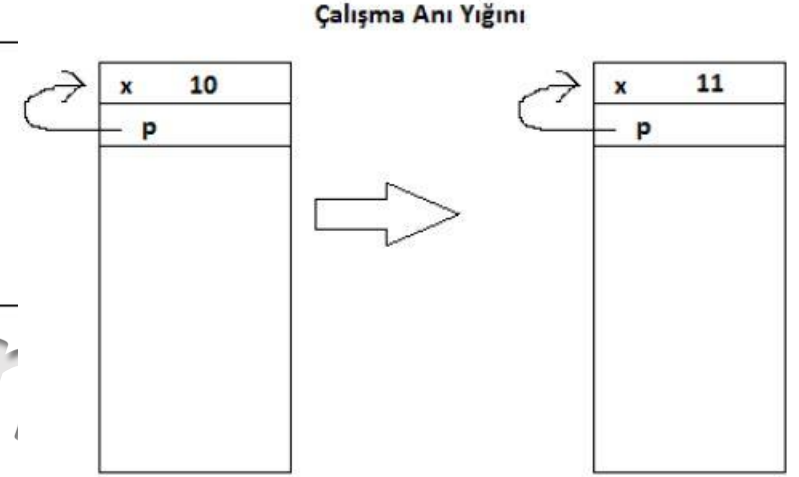
Arac, surucu'nun sahibidir (ownership).
Eğer önce Arac belleği silinseydi: ☞ İçindeki surucu adresine artık erişilemezdi ☞ Memory leak oluşurdu

Memory leak, heap'te ayrılan belleğin serbest bırakılmaması durumudur.

Bu doğrultuda C++ dilinde **göstericiler, herhangi bir bellek bölgesindeki bir değerin adresini gösterebilirler**. Gösterdikleri yer, statik bellek bölgesi, çalışma anı yığını ya da heap bellek bölgesi olabilir.

Kod 8:

```
int x=10;
int *p;
p=&x; // x değişkenin adresini tut
x++;
cout<<*p;
```



Göstericilerde iki önemli operatör bulunmaktadır. * ve & operatörü & operatörü adres operatörüdür.

Başına gelen değişken ya da göstericinin adresini getirir. * operatörü ise tutulan adresteki değeri getirir.

Yukarıdaki kod bloğunda bütün değişkenler çalışma anı yığnında tutulurlar. X bir değer barındıran bir değişken, p ise bir adres barındıran bir göstericidir. 3 satırda x'in adresi p göstericisine atanıyor. Daha sonra x değişkenin değeri bir arttırıldığı ve p göstericisi x'i gösterdiği için p ekrana yazdırıldığında x'in en son değeri yani 11 ekrana yazacaktır.

Kod 9:

```
int *p;  
int a=12;  
p=&a;  
  
cout<<*p;    // Gösterdiği yerdeki değeri yaz  
cout<<p;      // Gösterdiği yerin bellek adresi  
cout<<&p;     // Göstericinin kendi bulunduğu adres  
cout<<&a;     // a değişkeninin bulunduğu adres.
```

Göstericilerde atama işlemi sadece adres ataması olduğu için atanan gösterici de aynı adresi yani aynı değeri gösterecektir.

Kod 10:

```
int x=10;  
int *p,*r;  
p=&x;  
r=p;  
cout<<*r; // Ekrana 10 yazar.
```

Çalışma Anı Yığını

10
10
p
r

*p ile *r eşit

p ile r eşit değil

Adres İçerik

0x100 10 ← x

0x200 0x100 ← p

0x300 0x100 ← r

p ----\
----> x (10)
r ----/

Aşağıdaki örneğe bakıldığında x ve y farklı adreslerde tutulan iki değişken xp x'in adresini gösteren gösterici ve yp y'nin adresini gösteren gösterici. x=y dendiği zaman y'nin değeri x'in değerine atandığı için iki farklı adreste de 4 değeri vardır. Göstericileri ekrana yazdırdığınızda her ikisinde de 4 yazacaktır. Göstericilerin gösterdiği adreslerde bir değişiklik olmamıştır.

Kod 11:

```
int x=3,y=4;
int *xp,*yp;
xp=&x;
yp=&y;
x=y;
cout<<*xp<<endl;
cout<<*yp<<endl;
```

Gösterici Göstericileri

Bu tip göstericilerin diğer göstericilerden farkı, **gösterdikleri yerde yine bir gösterici bulunmaktadır**. Yani **değere iki adımda** erişilebilmektedir.

`int *p; // Bir tamsayı değerinin adresini gösteren gösterici`

`int **pp; // Bir tamsayı göstericisini gösteren gösterici.`

Aşağıdaki örneğe bakıldığında a bir tamsayı değişkenidir, p ise a'yı gösteren bir gösterici, pp'de p'yi gösteren bir gösterici yani gösterici göstericisidir (pointer to pointer).

Çalışma Anı Yığını

0x28ff1c	a	10
0x28ff18	p	0x28ff1c
0x28ff14	pp	0x28ff18

Kod 14:

```
int a=10;  
int *p = &a;  
int **pp = &p;  
cout<<*p; // 10 yazar  
cout<<**pp; // 10 yazar
```

```
cout<<*pp; // a'nın adresi  
cout<<pp; // p'nin adresi  
cout<<&pp; // pp'nin adresi
```

AKALIN

Dr.

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int *p = &a;
    int **pp = &p;

    cout << "Baslangic:\n";
    cout << "a = " << a << endl;

    *p += 3;

    cout << "\n*p += 3 sonrasi:\n";
    cout << "a = " << a << endl;

    **pp += 2;

    cout << "\n**pp += 2 sonrasi:\n";
    cout << "a = " << a << endl;

    return 0;
}
```

tma AKALIN

```
#include <iostream>
using namespace std;
int main() {
    int a = 5;
    int b = 50;

    int *ptr = &a;
    int **pptr = &ptr;

    cout << "Baslangic:\n";
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    **pptr += 3;

    ptr = &b;

    *ptr -= 20;

    **pptr += 10;

    cout << "\nSon durum:\n";
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    return 0;
}
```

KOD 1:

```
#include <iostream>
using namespace std;

struct Node {
    struct Node *p;
    struct Node *r;
    int eleman;
    Node(int eleman) : eleman(eleman),
p(nullptr), r(nullptr) {} // Yapıcı fonksiyon
tanımı eklendi
};

int main() {
    int x = 17;
    int *xptr = &x;
    Node *p = new Node(x); // Node tipinde bir
nesne oluşturuldu
    (*xptr)++;
    Node *r = new Node(x); // Node tipinde bir
nesne oluşturuldu
```

```
Node **ptr = &r; // Node tipinde bir pointerın
pointerı

if (ptr == &r)
    (*xptr)++;

if (*ptr == r)
    (*xptr)++;

cout << x << endl;
cout << *xptr << endl;
cout << p->eleman << endl;
cout << r->eleman << endl;
cout << (*ptr)->eleman << endl;

delete r;
delete p;

return 0;
}
```

İlk 5 soruyu yukarıda verilen koda göre cevaplayınız.

1-) KOD 1 isimli kod bloğunda `cout << x << endl;` kod satırının çıktısı aşağıdaki seçeneklerden hangisinde doğru verilmiştir?

A) 17 B) 18 C) 0x61ff00 D) 20 E) 0x61fefc

2-) KOD 1 isimli kod bloğunda `cout << *xptr << endl;` kod satırının çıktısı aşağıdaki seçeneklerden hangisinde doğru verilmiştir?

A) 17 B) 18 C) 0x61ff00 D) 20 E) 0x61fefc

3-) KOD 1 isimli kod bloğunda `cout << p->eleman << endl;` kod satırının çıktısı aşağıdaki seçeneklerden hangisinde doğru verilmiştir?

A) 17 B) 18 C) 0x61ff00 D) 20 E) 0x61fefc

4-) KOD 1 isimli kod bloğunda `cout << r->eleman << endl;` kod satırının çıktısı aşağıdaki seçeneklerden hangisinde doğru verilmiştir?

A) 17 B) 18 C) 0x61ff00 D) 20 E) 0x61fefc

5-) KOD 1 isimli kod bloğunda `cout << (*ptr)->eleman << endl;` kod satırının çıktısı aşağıdaki seçeneklerden hangisinde doğru verilmiştir?

A) 17 B) 18 C) 0x61ff00 D) 20 E) 0x61fefc

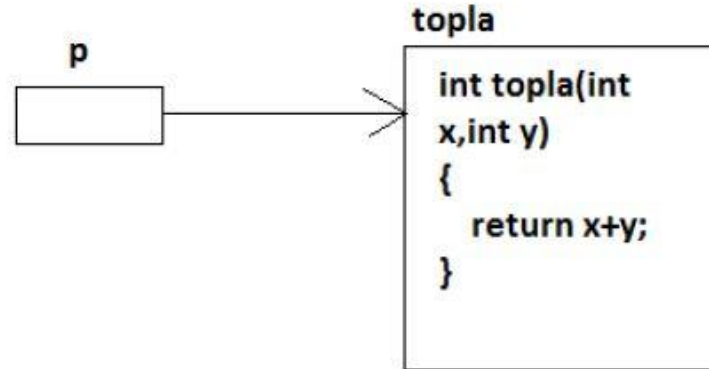
Fonksiyon Göstericileri

Göstericiler, ilkel türleri ve tanımlamış olduğunuz nesneleri gösterebilecekleri gibi, **bir fonksiyona da işaret edebilirler**. Bu tip göstericilerde en önemli husus, **gösterici ile gösterdiği fonksiyonun prototipi uymalıdır**. Örneğin `int (*p)(int,int);` şeklinde tanımlanmış bir fonksiyon göstericisi, işaret edebileceği fonksiyon iki parametre alan ve tamsayı döndüren bir fonksiyon olabilir. Göstericideki **parantez**, derleyici tarafından **normal göstericiymiş** gibi ya da **gösterici döndüren bir fonksiyonmuş gibi algılanmasın** diye kullanılmıştır.

Kod 15:

```
int topla(int x,int y)
{
    return x+y;
}
int main()
{
    int (*p)(int,int);
    p=topla;
    cout<<(*p)(25,30);
}
```

p işaretçisi artık topla fonksiyonunun adresini tutuyor.
Yani: topla → bir fonksiyon
p → o fonksiyonun adresini tutan pointer



Yandaki örneğe
bakıldığında p
fonksiyon
göstericisi
tanımlanıyor. p
fonksiyon
göstericisi topla
fonksiyonunu
göstermesi
sağlanıyor daha
sonra p göstericisi
çağrılıyor.

AKALIN

Void Gösterici

Türü olmayan bir göstericidir. Dolayısıyla yeri geldiğinde bir **tamsayıyı** gösterebileceği gibi yeri geldiğinde bir **ondalık** sayıyı da gösterebilir. Sadece göstericinin **gösterdiği yer kullanılacağı** zaman, derleyicinin o **anki hangi tür olduğu bilmesi açısından dönüştürme işlemi uygulanmalıdır.** Bunun örneği aşağıdaki kod parçasında görülebilir.

Kod 17:

```
int x=100;
float a=12.5;
void* obj;
obj=&x;
cout<<*static_cast<int *>(obj)<<endl;
obj=&a;
cout<<*static_cast<float *>(obj)<<endl;
```

int adresini int*'e, float adresini float*'e çevirmek güvenlidir. float adresini int*'e çevirmek veya tam tersini yapmak yanlış ve tehlikelidir.

void*: Herhangi bir veri tipinin adresini tutabilen genel amaçlı pointer'dır.

obj artık x'in adresini tutuyor

Ama tipi void*

Bu yüzden: tekrar int*'e çeviriyoruz

Sonra * ile değeri okuyoruz

Dinamik Bellek Yönetimi

Bir program içerisinde **nesneler** ve **yerleşik tipler** veya **kullanıcı tanımlı tiplerin dizileri** için **belleğin ayrılmasını ve serbest bırakılmasını kontrol** edebilirsiniz. Bu, **dinamik bellek yönetimi** olarak bilinir ve **new ve delete** operatörleri ile gerçekleştirilir.

Çalışma zamanında, bir nesneyi veya yerleşik bir diziyi tutabilecek miktarda **belleği new operatörü kullanarak dinamik olarak ayırabilirsiniz**. Nesne veya yerleşik dizi boş alanda dinamik olarak yer ayrılacak nesnelere depolaması için her bir programa atanmış bellek bölgesi oluşturulacaktır. **Boş alanda bellek ayrıldığında new operatörü tarafından geriye döndürülen işaretçi ile ona erişebilirsiniz**. Belleğe ihtiyacınız kalmadığında serbest bırakmak için onu boş alana **delete operatörünü kullanarak geri döndürebilirsiniz**. Daha sonra bu alan gelecekteki işlemler için yeniden kullanılabilir

New ile dinamik bellek atama

```
Time *timePtr=new Time();
```

new operatörü, Time tipindeki bir nesne için uygun boyutta bir alanı ayırır, **nesneyi ilklendirmek için varsayılan kurucu çağırır** ve new operatörünün sağında bildirilmiş olan tipe (örn., Time*) **bir işaretçi döndürür**. Eğer new, **nesne için bellekte yeterli alan bulamazsa istisna fırlatarak hata oluştuğunu bildirir**.

delete ile dinamik belleği serbest bırakma

Dinamik olarak yer **ayrılmış bir nesneyi yok etmek ve nesne için ayrılmış alanı boşaltmak için delete operatörü** aşağıdaki gibi kullanılır.

delete timePtr;

Bu ifade öncelikle, timePtr tarafından işaret edilen **nesnenin yıkıcısını çağırır ve daha sonra nesne ile ilişkilendirilmiş belleği** serbest bırakır.

Böylelikle **bellek boş alana** geri verilmiş olur.

Dinamik belleği ilklendirme

Yeni oluşturulmuş temel tipe ait bir değişken için aşağıda görüldüğü şekilde bir ilklendirici sağlayabiliriz.

```
Double *ptr=new double(3.14159);
```

Bu ifade yeni oluşturulmuş double tipine ve 3.14159 değerine sahip bir değişkeni ilklendirir ve elde edilen işaretçiye ptr değişkenine atar. Aynı söz dizimi **bir nesnenin kurucusuna virgülle ayrılmış bir argüman listesi bildirmek** için de kullanılabilir. Örneğin;

```
Time *timePtr=new Time(12,45,0);
```

Yerleşik dizileri new[] ile Dinamik olarak Ayırma

new operatörü, yerleşik **dizileri dinamik olarak ayırmak** için de kullanılabilir. Örneğin 10 elemanlı bir tamsayı dizi aşağıdaki ifade ile ayrılabilir ve gradesArray değişkenine atanabilir.

```
int *gradesArray=new int[10]();
```

Bu ifade, **tamsayı işaretçi tipinde gradesArray isminde bir değişken bildirir ve ona dinamik olarak ayrılmış 10 elemanlı bir tamsayı dizisinin ilk elemanına işaret eden işaretçiyi atar.** new int[] değerini takip eden parantezler, dizinin elemanlarını ilgilendirir- **temel nümerik tipler sıfır değerine ayarlanır, bool tipler false değerini ayarlanır, işaretçiler nullptr değerini ayarlanır ve sınıf nesneleri varsayılan kurucuları ile ilklendirilir.**

Dinamik olarak Ayrılmış Yerleşik Dizileri delete[] ile Serbest Bırakma

gradesArray tarafından işaret edilen **belleği serbest bırakmak** için
aşağıdaki ifadeyi kullanırız

delete [] gradesArray;

Eğer yukarıdaki ifade köşeli parantezleri **[] içermezse** ve gradesArray bir
nesne dizisine işaret ederse **sonuç tanımsız** olur.

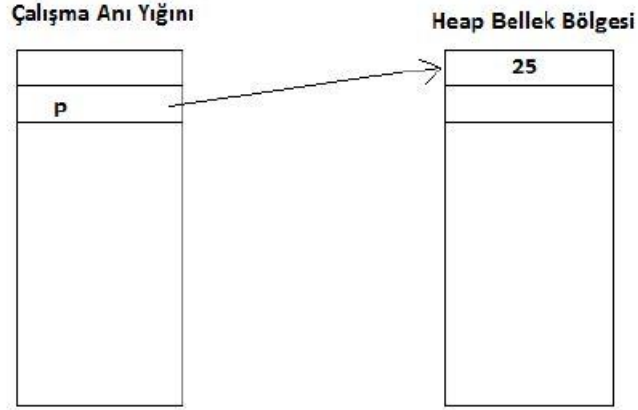
Statik bellek bölgesi, çalışma anı yığını ya da heap bellek bölgesi kapsamında **new operatörünü** inceleyelim.

Programcının kontrolüne bırakılmış olan **heap bellek bölgesinde**, bellekten yer ayırmak için **new** operatörü kullanılır. Programın herhangi bir yerinde **new operatörü görülmüş ise mutlaka orada heap bellek bölgesinden bir yer ayırma vardır. Bu ayrılan alan gösterici kaybedilmeden geri döndürülmelidir.** Yoksa ayrılmış olan o alan **bilgisayar kapanan kadar** kullanılamaz bir alan olacak yani **çöp** olayı dediğimiz durum gerçekleşecektir.

```
int *p = new int();  
int *p = new int(100);
```



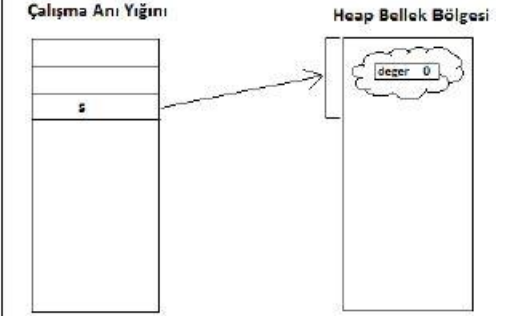
*p=25; // p nin gösterdiği adresteki yere 25 değerini yaz.



Heap bellek bölgesinde herhangi bir tür veri için yer ayrılabilir.
Yandaki kod bloğuna bakılabilir.

Kod 18:

```
class Sayi{  
    private:  
        int deger;  
    public:  
        Sayi():deger(0) { }  
};  
  
Sayi *s = new Sayi();
```



Yukarıdaki kod bloğunda Sayi nesnesini heap bölgesinde oluşturan kişi onu kullandıktan sonra belleğe geri döndürecektir.

Aşağıdaki örnekte de verilen durumda oluşturulan bir nesne içerisinde **bir veya daha fazla nesne heap bellek bölgesinde nesnenin kendisinin oluşması ile birlikte oluşabilir**. Bu durumda sınıfı tasarlayanların görevi olarak nesnenin yıkılma anında yani yıkıcı metot içerisinde bu heap'te oluşan nesneleri geri döndürmelidirler.

Kod 19:

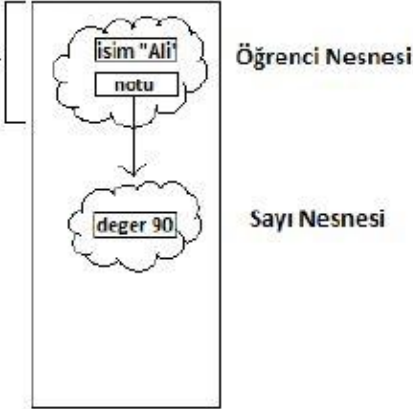
```
class Ogrenci{
private:
    string isim;
    Sayi *notu;
public:
```

- Eğer nesne sadece o fonksiyon içinde kullanılacaksa ve boyutu çok büyük değilse **Sayi s1**; kullanmak en güvenli yoldur.
- Eğer nesnenin fonksiyon bittikten sonra da yaşamasını istiyorsan veya çok büyük miktarda veri tutacaksan **new** anahtar kelimesini kullanmalısın.

Çalışma Anı Yığını



Heap Bellek Bölgesi



```
Ogrenci(string ad,int nt){
    isim=ad;
    notu=new Sayi(nt);
}
~Ogrenci(){ // Yıkıcı metot
    delete notu;
}

};

int main(){
    Ogrenci *ogr = new Ogrenci("Ali",90);
    delete ogr;
    return 0;
}
```

Heap bellek bölgesinde oluşturulmuş bir alanı geri döndürmek için **delete** komutu kullanılır. **delete sadece göstericilerde** kullanılabilecek bir komuttur.

Sayi s1;
Sayi *sp = new Sayi();
delete s1; // **Hatalı kullanım**
delete sp; // **Doğru kullanım**

- s1 nesnesi stack bellekte(çalışma anı yığıtı) oluşturulur.
- Program s1 için new kullanmaz, yani elle delete çağırmaya gerek yoktur.
- s1 nesnesi, fonksiyonun scope'undan çıkınca otomatik olarak yok edilir.

Sayi s1; // **1** Statik (otomatik) bellek tahsisi
Sayi *sp = new Sayi(); // **2** Dinamik bellek tahsisi

Çöp olayını daha iyi anlayabilmek açısından aşağıdaki örnekte **Heap bellek bölgesinde oluşturduğumuz Sayı nesnesini for döngüsünün sonunda geri döndürmediğimiz için *s göstericisi de for döngüsü içinde tanımlandığı için bir döngü bittiğinde gösterici yok edilecektir** ama gösterdiği bölge programcıya ait olduğu için ve göstericisini kaybettiği için orası **çöp bölgeye dönmüştür**. Ve for **döngüsü ile adresler yazdırıldığında görülecektir ki sürekli farklı adresten** yer ayrılmaktadır.

Kod 20:

```
for(int i=0;i<10;i++){  
    Sayi *s = new Sayi(125);  
    cout<<s<<endl;  
}
```

0x7b1838
0x7b2458
0x7b2468
0x7b2478
0x7b2488
0x7b2498
0x7b24a8
0x7b24b8
0x7b24c8
0x7b24d8

Kod 21:

```
for(int i=0;i<10;i++){  
    Sayi *s = new Sayi(125);  
    cout<<s<<endl;  
    delete s;  
}
```

0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838
0x4d1838

Doğru olan yapıldığında yani **her döngünün sonunda göstericiyi kaybetmeden gösterdiği yeri belleğe geri iade ettiğimizde** ekrana adresleri yazdırırsak sürekli aynı adresin bize verildiğini görürüz. Bunun nedeni o alanın geri döndürüldüğü için tekrar kullanılabilir olmasıdır.

Sallanan Göstericiler (Dangling Pointers)

Sallanan göstericiler (dangling pointers), artık geçerli olmayan bir bellek adresini gösteren göstericilerdir. Yani gösterici, önceden var olan bir nesneye veya bellek bölgesine işaret eder, ancak bu nesne/bellek artık geçerli değildir (silinmiş veya fonksiyon tamamlandığında kaybolmuş olabilir).

1- Serbest Bırakılmış (Silinmiş) Belleğe İşaret Etme

2- Otomatik (Yerel) Değişkenin Adresini Döndürme

3- Geçersiz Bellek Alanlarına İşaret Etme

1-Eğer bir gösterici, delete veya free ile serbest bırakılan belleğe hâlâ işaret ediyorsa, sallanan bir gösterici olur.

2-Bir fonksiyon içinde tanımlanan bir yerel değişkenin adresi fonksiyon sona erdiğinde geçersiz olur. Eğer bu adres bir göstericiye atanırsa, sallanan gösterici oluşur.

3-Bir gösterici geçersiz bir belleğe işaret ediyorsa (örneğin, bir dizinin dışına çıkarsa veya yanlış bellek erişimi yapılırsa), sallanan gösterici oluşur.

Sallanan Göstericileri Önleme Yöntemleri

1-Silinen belleği gösteren göstericileri nullptr yapmak

2-Yerel değişkenlerin adresini döndürmekten kaçınmak

3-Geçersiz bellek erişimine dikkat etmek

Bellek Sızıntısı (Memory Leak) Nasıl Oluşur?

Programın new ile ayırdığı belleğin adresini kaybedip, o belleği delete ile geri vermemesi durumudur. Bellek hâlâ doludur ama artık o belleğe ulaşabileceğimiz bir pointer yoktur

Basit Örnek

```
int* p = new int(5);  
p = nullptr; // Bellek sızıntısı
```

Ne oldu?

new int(5) bellekte yer ayırdı.

Ama p = nullptr dediğimiz anda o adresi kaybettik.

delete edemiyoruz çünkü artık adres elimizde değil.

Özetle Bellek sızıntısı oluştu.

Bellek sızıntısı kapsamında farklı senaryoları derste ifade etmiştim. Burada ayrı başlık altında sunmak istedim.

Tanımsız Davranış (Undefined Behavior) Nedir?

Tanımsız davranış, C++ standardının

“Bu durumda programın ne yapacağı garanti edilmez.”
dediği durumdur.

Yani sonuç **belirsizdir**.

Program:

- Çalışabilir
- Çökebilir
- Farklı sonuç üretebilir
- Farklı derleyicide farklı davranabilir

Hiçbir şey garanti değildir.

Özetle, program, standardın izin vermediği bir şey yaparsa sonuç kontrol dışına çıkar.

ÖRNEKLER

1- Silinmiş belleğe erişmek

```
int* p = new int(5);  
delete p;  
cout << *p; // Tanımsız davranış
```

2- Dizi sınırını aşmak

```
int a[3] = {1,2,3};  
cout << a[5]; // Tanımsız davranış
```

3- Başlatılmamış değişken kullanmak

```
int x;  
cout << x; // Tanımsız davranış
```

4-Null pointer dereference

```
int* p = nullptr;  
cout << *p; // Tanımsız davranış
```

Neden Tehlikelidir?

Çünkü: Derleyici hata vermez

Runtime hata vermeyebilir.

Bazen doğru çalışıyormuş gibi görünür

Production'da patlayabilir

Bir sonraki hafta da yaygın olarak kullanılan
veri yapıları algoritmaları anlatılacaktır...

Kaynakça

Fatih Adak Lab 2 Notları

(<https://ahmetarslan.sakarya.edu.tr/sites/ahmetarslan.sakarya.edu.tr/file/Hafta-2-Gostericiler.pdf>)

Veri Yapıları ve Algoritmalar, 12. Baskı, Rifat Çölkesen

Nejat Yumuşak, fatih Adak Veri C++ ile veri yapıları, 3.baskı

C++ ile programlama, Dokuzuncu baskıdan çeviri, Palme Yayınları, Pau Deitel, Harvey Deitel, Çeviri Editörü Cemil Öz

Veri Yapıları, Hakan Kutucu

Algoritma ve Programlamaya giriş, Fahri Vatansever

Programlama ve Veri Yapılarına Giriş C,C++ ve Java ile, Sadi Evren Şeker

<https://chatgpt.com/>